Videojuegos Disada

Videojuegos Disada

Proyecto Final — Desarrollo de una App con Flutter

Integrantes del Equipo

Daniel Martínez Sánchez • José Salvador Montes Luna • Diego Eduardo Hurtado Hernández

AGENDA

01 Resumen del Proyecto

02 Arquitectura y Estructura

03 Flujo de Navegación

04 Diseño de la UI

05 Consumo de API (RAWG)

06 Persistencia de Datos

07 Animaciones

08 Retos Técnicos

09 Conclusiones y Mejoras

RESUMEN DEL PROYECTO

Game Vault — Videojuegos Disada es una aplicación móvil desarrollada en Flutter que simula una tienda en línea de videojuegos. Permite explorar catálogos por género, gestionar favoritos y completar un proceso de compra completo.

Público Objetivo

- Gamers 15–35 años
- Compradores casuales
- Fans del dark theme

Plataforma

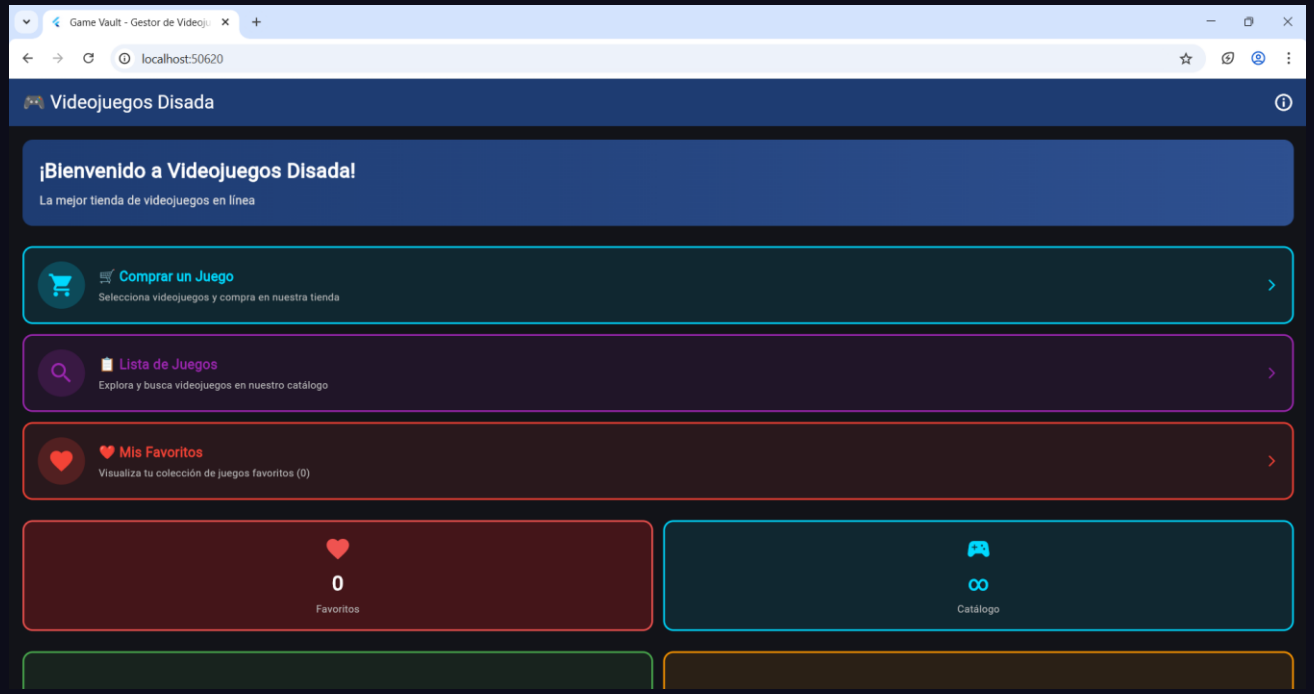
- Android (Flutter)
- Dark Gaming Theme
-

Materia

- Desarrollo Móvil Flutter
- Universidad Cuauhtémoc
- Abril 2026

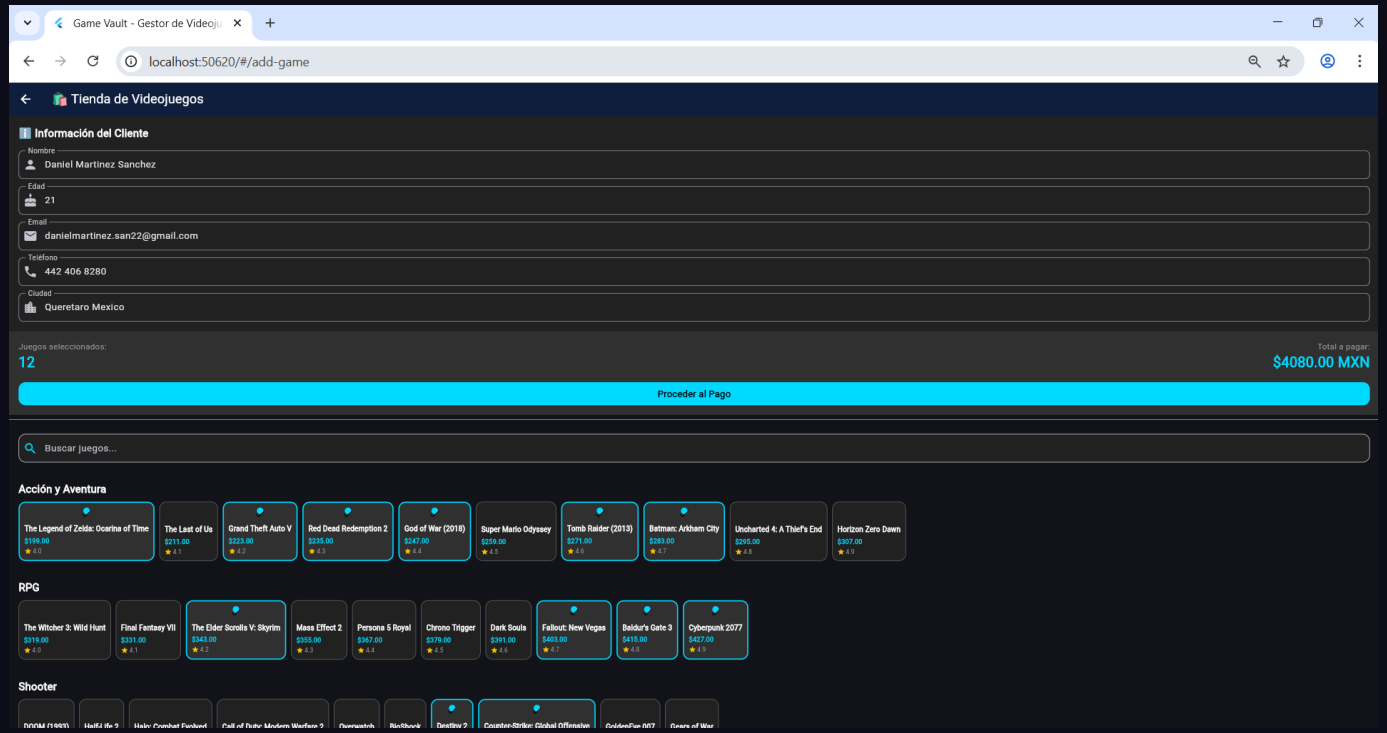
Diseño de la Interfaz de la Tienda de Videojuegos

Pantalla 1 : Home Screen Inicio Banner de bienvenida, tres accesos directos Comprar, Lista de Juegos, Mis Favoritos) y cuatro tarjetas de estadísticas con bordes de colores.



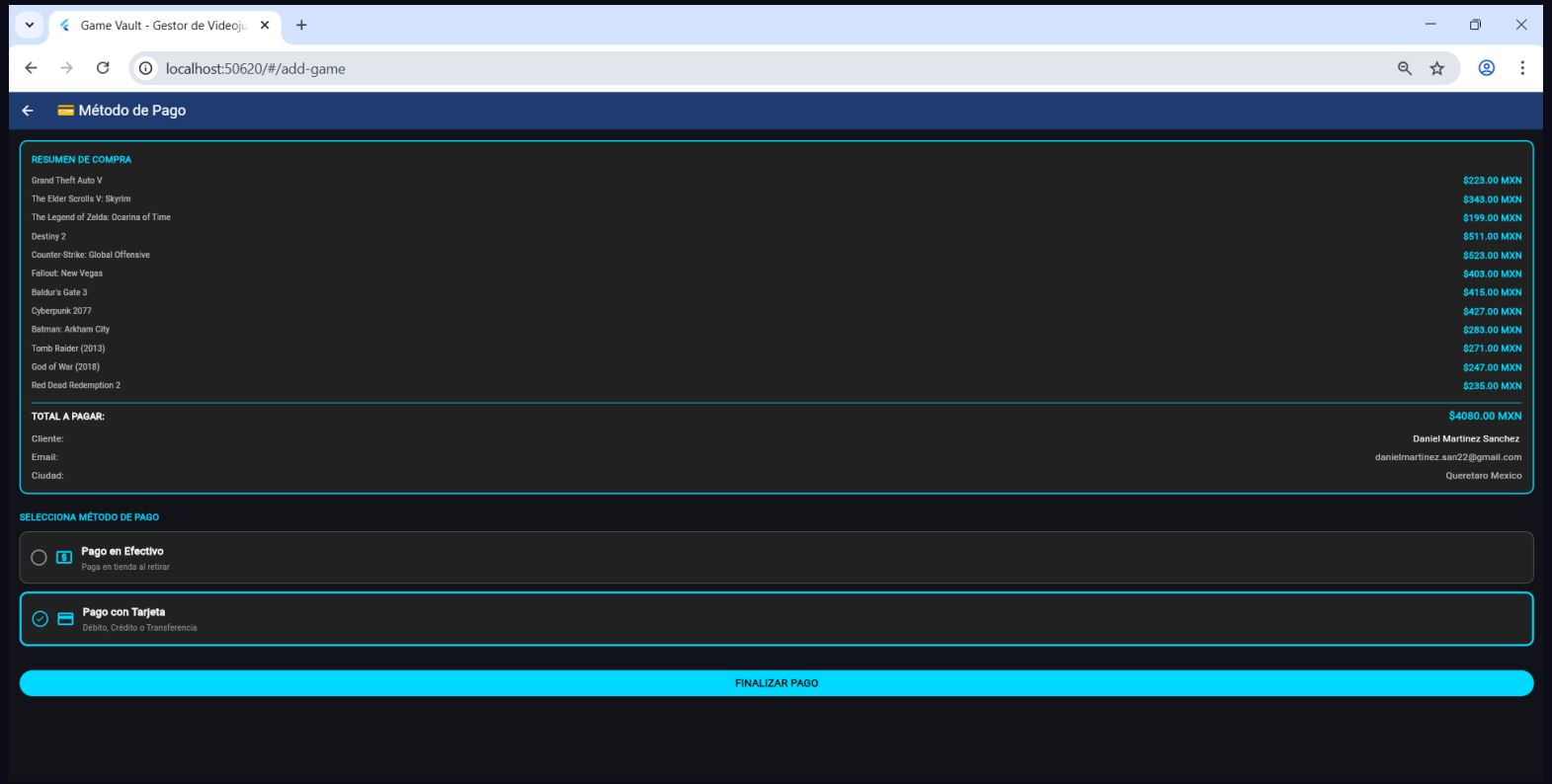
Comprar en la Tienda de Videojuegos

Pantalla 2 : Tienda de Videojuegos Formulario de datos del cliente en la parte superior, catálogo de juegos agrupado por géneros con scroll horizontal.



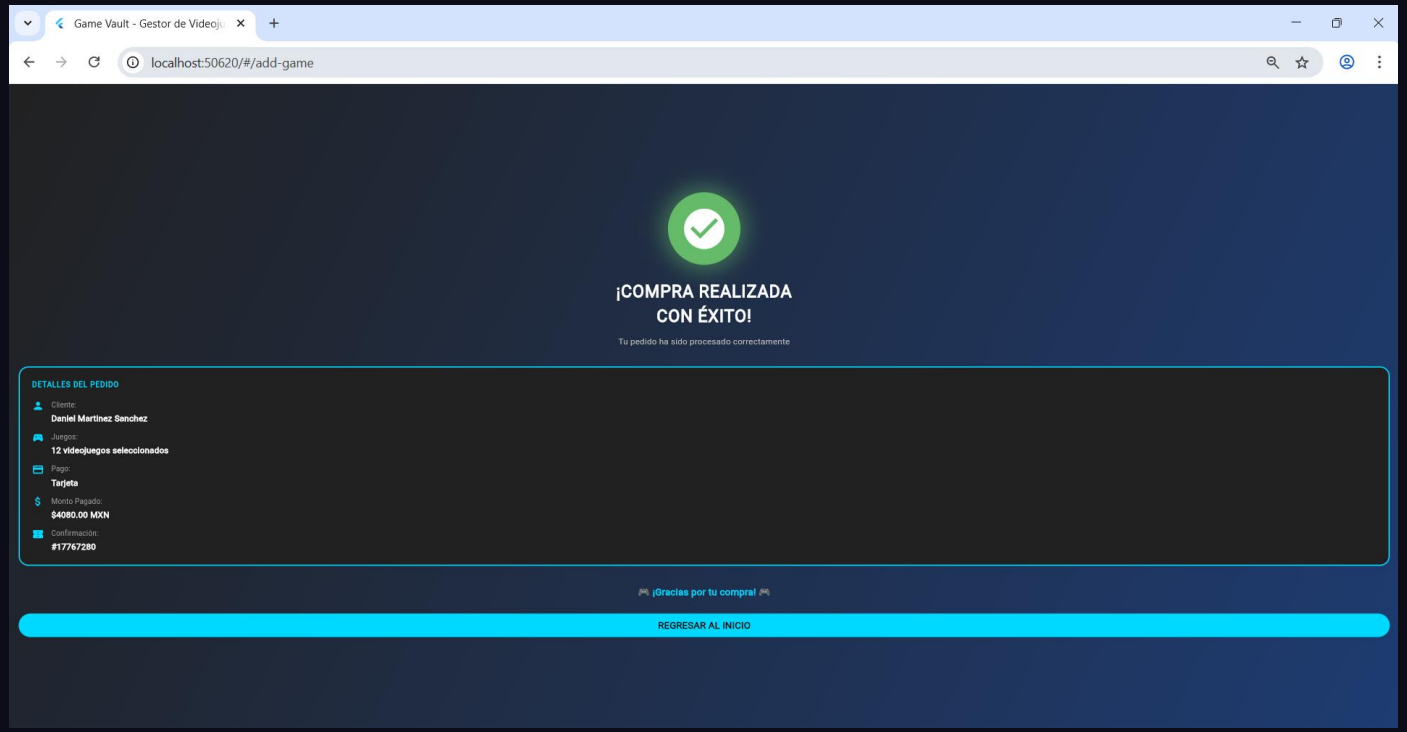
Metodo de Pago

Pantalla 3 : Método de Pago Resumen detallado de todos los juegos seleccionados con precios individuales, total a pagar y datos del cliente. Permite elegir entre Pago en Efectivo o Pago con Tarjeta.



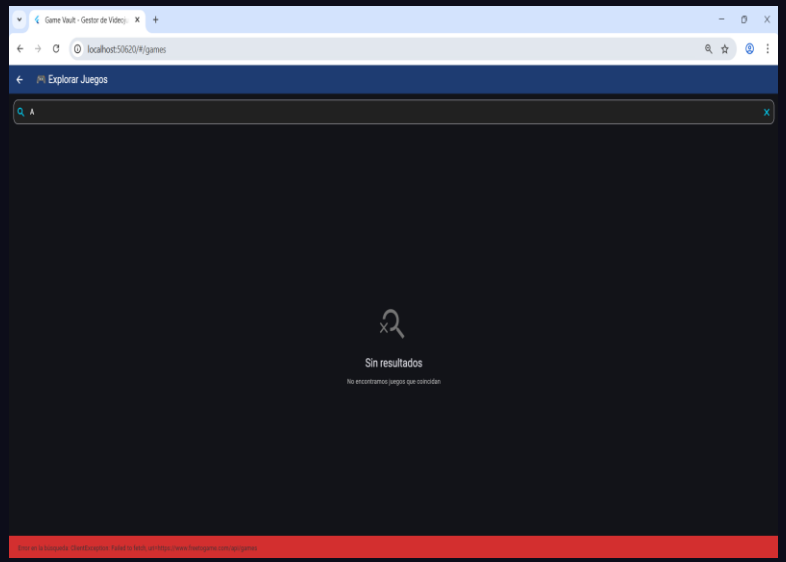
La Compra Realizada

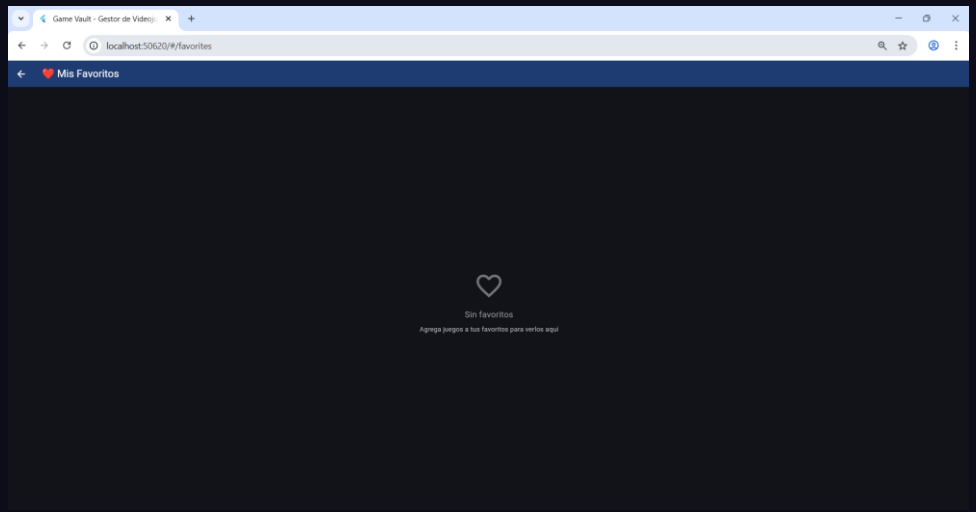
Pantalla 4 : Compra Exitosa Confirmación con ícono verde animado (ScaleTransition). Muestra nombre del cliente, número de juegos, método de pago, monto y número de confirmación aleatorio.



Explorador de Juegos y Favoritos

Pantalla 5: Explorador de Juegos Catálogo completo con buscador en tiempo real. Paginación infinita con botón Cargar más. Muestra estado vacío ilustrativo cuando no hay resultados.





PRINCIPALES FUNCIONALIDADES

Home Screen

Accesos directos y estadísticas en tiempo real con animaciones stagger

Tienda

Catálogo con 50+ juegos por género, precios en MXN y calificaciones

Proceso de Compra

Formulario del cliente, selección múltiple y confirmación con número de pedido

Explorador

Buscador en tiempo real con paginación infinita desde API RAWG

Mis Favoritos

Lista persistida con Hive, ordenamiento por nombre, calificación y año

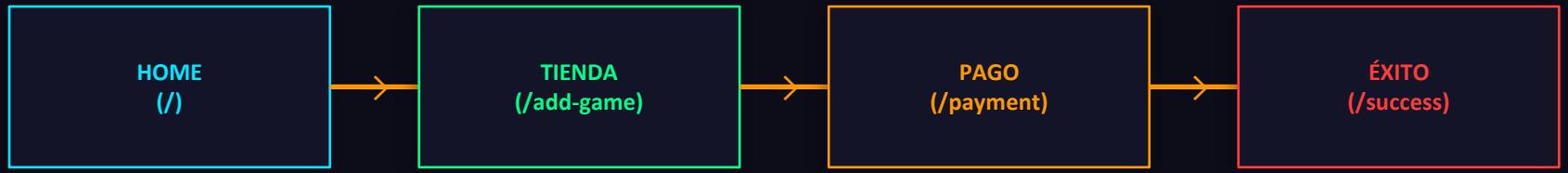
Detalle de Juego

Imagen, descripción, géneros y lanzamiento via ApiService

ARQUITECTURA — ESTRUCTURA DE CARPETAS

`lib/main.dart`	Punto de entrada. Inicializa Hive, SharedPreferences y rutas nombradas
`lib/screens/home_screen.dart`	Pantalla principal con bienvenida, accesos directos y estadísticas
`lib/screens/add_game_screen.dart`	Tienda con formulario de cliente y catálogo por géneros
`lib/screens/payment_screen.dart`	Pantalla de método de pago con resumen de compra
`lib/screens/games_list_screen.dart`	Explorador del catálogo completo con búsqueda y paginación
`lib/screens/favorites_screen.dart`	Lista de favoritos con ordenamiento múltiple
`lib/models/game.dart`	Modelo Game con adaptador Hive (HiveType, HiveField)
`lib/services/game_storage.dart`	GameStorage: capa de acceso a Hive y SharedPreferences
`lib/services/api_service.dart`	ApiService: peticiones HTTP a la API RAWG

FLUJO DE NAVEGACIÓN



Flujos Secundarios

```
HOME (/) → Lista de Juegos (/games) → [back] HOME
```

```
HOME (/) → Mis Favoritos (/favorites) → [back] HOME
```

```
TIENDA (/add-game) → Detalle de Juego (/game-detail) → [back] TIENDA
```

Tecnología: Named Routes con onGenerateRoute — paso de argumentos complejos entre pantallas

DISEÑO DE LA UI — WIDGETS CLAVE

ListView

Home, Tienda, Favoritos

Scrolling vertical de listas y catálogo por género

GridView

Tienda, Lista de Juegos

Cuadrícula deslizable de juegos con 2 columnas

TextField + Controller

Tienda, Explorador

Formulario del cliente y buscador en tiempo real

StatefulWidget + setState

Todas

Actualiza la UI al seleccionar juegos o favoritos

AnimatedContainer

Tienda

Animación de selección en tarjetas de juego

RadioListTile

Pago

Selección excluyente de método de pago

PopupMenuButton

Favoritos

Menú desplegable para ordenar por nombre, calificación o año

ScaleTransition

Compra Exitosa

Animación de escala en ícono de confirmación

PANTALLA 1 — HOME SCREEN

Banner de Bienvenida

- Banner animado de bienvenida con nombre de la app
- Acceso rápido a la Tienda de Videojuegos
- Explorador del catálogo completo con búsqueda
- Mis Favoritos — lista persistida con Hive
- 4 tarjetas de estadísticas con AnimatedOpacity (stagger 150ms)
- Dark theme gaming con colores de acento por categoría

PANTALLA 2 — TIENDA DE VIDEOJUEGOS

Catálogo por Géneros

- Formulario del cliente: nombre, email, teléfono, ciudad
- 50+ juegos distribuidos en géneros (Acción, RPG, Shooter, Plataforma, Puzzle)
- ↔ GridView horizontal deslizable por género
- AnimatedContainer con borde cyan al seleccionar juego
- Carrito visible: cantidad seleccionada y total en tiempo real
- Validación de formulario antes de avanzar al pago

PANTALLAS 3 & 4 — PAGO Y CONFIRMACIÓN

Método de Pago

The Last of Us — $849

God of War — $699

Elden Ring — $999

GTA V — $499

Total: $3,046 MXN

- **Pago en Efectivo**
- **Pago con Tarjeta**

FINALIZAR COMPRA

COMPRA REALIZADA

✓

¡CON ÉXITO!

👤 Daniel Martínez Sánchez

🎮 14 videojuegos seleccionados

💳 Pago con Tarjeta

💰 $3,046.00 MXN

🔑 Confirmación: #177666

ScaleTransition: escala 0→1.0 en 600ms con Curves.elasticOut

PANTALLAS 5 & 6 — EXPLORADOR Y FAVORITOS

Explorar Juegos

Buscar juegos...

The Witcher 3 ⭐ 4.7

Red Dead Redemption ⭐ 4.8

Cyberpunk 2077 ⭐ 4.2

Halo Infinite ⭐ 4.1

Minecraft ⭐ 4.9

Cargar más...

Mis Favoritos

Ordenar ▼

Sin favoritos
Agrega juegos a tus favoritos

Ordenar por: Nombre | Calificación | Año
Acción: PopupMenuButton con checkmarks cyan

CONSUMO DE API — RAWG VIDEO GAMES

Nombre	RAWG Video Games Database API
URL Base	`https://api.rawg.io/api`
Endpoint	`https://api.rawg.io/api/games?key={KEY}&page_size=20&page={n}`
Método HTTP	GET
Respuesta JSON	id, name, genres[], rating, background_image, released
Librería Flutter	http (^1.1.0) — paquete oficial para peticiones HTTP en Dart

Manejo de Errores

- CircularProgressIndicator inicial
- Indicador inferior al cargar más
- try/catch con mensaje amigable
- Estado vacío con ícono ilustrativo
- Timeout: 15 segundos máximo por petición
- Caché offline con Hive

PERSISTENCIA — SHARED PREFERENCES

`last_customer_name`	Recuerda el nombre del cliente para autocompletar el formulario
`last_customer_email`	Email del cliente para autocompletar en la siguiente sesión
`last_customer_city`	Ciudad del cliente para agilizar futuras compras
`preferred_payment`	Último método de pago seleccionado (efectivo/tarjeta)
`total_purchases_count`	Contador de compras mostrado como estadística en el Home

Nota: SharedPreferences se usa para datos simples clave-valor. Para objetos complejos (juegos, pedidos) se usa Hive.

PERSISTENCIA DE DATOS — HIVE

favoritesBox

Datos: Lista de objetos Game (HiveObject)

Propósito: Persiste favoritos del usuario entre sesiones

Clase: GameStorage → lib/services/game_storage.dart

purchasesBox

Datos: Historial de Pedidos con juegos, total y método de pago

Propósito: Registro histórico de todas las compras realizadas

Clase: GameStorage → lib/services/game_storage.dart

gamesBox

Datos: Catálogo cacheado desde la API RAWG

Propósito: Acceso offline al catálogo descargado previamente

Clase: GameStorage → lib/services/game_storage.dart

Inicialización en main.dart

`await gameStorage.init()` → Se llama antes de `runApp()` garantizando que la BD esté lista antes de lanzar la app.

ANIMACIONES EN FLUTTER

01

AnimatedContainer

Tienda — Selección de Juego

Al tocar una tarjeta, cambia color de fondo y grosor del borde con Curves.easeInOut en 200ms. Los juegos seleccionados muestran borde cyan brillante.

```
duration: 200ms | Curves.easeInOut | border: 2px cyan
```

02

ScaleTransition + AnimationController

Pantalla de Compra Exitosa

El ícono de verificación verde crece desde escala 0 hasta 1.0 en 600ms con Curves.elasticOut, creando un efecto de 'pop' satisfactorio al confirmar la compra.

```
duration: 600ms | Curves.elasticOut | scale: 0 → 1.0
```

03

AnimatedOpacity + Stagger

Home Screen — Estadísticas

Las cuatro tarjetas de estadísticas aparecen gradualmente con un stagger de 150ms entre cada una, evitando que todos los elementos aparezcan simultáneamente al cargar la pantalla.

```
stagger: 150ms entre cards | opacity: 0 → 1
```

DEPENDENCIAS — pubspec.yaml

hive ^2.2.3

Base de datos local NoSQL para favoritos e historial

hive_flutter ^1.1.0

Integración nativa de Hive con Flutter Widget lifecycle

shared_preferences ^2.2.2

Almacenamiento de preferencias simples del usuario

http ^1.1.0

Peticiones HTTP para la API RAWG (Dart oficial)

provider ^6.1.1

Gestión de estado global — carrito de compras

flutter_launcher_icons ^0.13.1

Generación del ícono de app en todas las resoluciones

build_runner ^2.4.7

Generación de código — adaptadores Hive TypeAdapter

hive_generator ^2.0.1

TypeAdapter autogenerado para los modelos Hive

RETOS TÉCNICOS Y SOLUCIONES

01

Gestión del Estado en la Tienda

⚠ Mantener sincronizado el estado de selección de múltiples juegos entre pantallas y al regresar desde pago.

✓ onGenerateRoute en main.dart pasando argumentos: List<Map<String,dynamic>>, datos del cliente, totalItems y totalPrice sin estado global.

02

Null Safety con Objetos Hive

⚠ Objetos recuperados con operador ?. generaban warnings de null-aware innecesarios al mapear HiveObject → Game.

✓ Operador ?? con valores por defecto en cada campo: fav?.name ?? '', fav?.rating ?? 0.0, garantizando null safety sin excepciones.

03

Paginación Infinita RAWG API

⚠ La API RAWG devuelve resultados paginados; cargar todo el catálogo en una petición agotaba límites y degradaba rendimiento.

✓ Paginación con _currentPage, _isLoadingMore y _hasMoreGames. Al detectar scroll final o 'Cargar más', incrementa _currentPage y acumula en _games.

04

Diseño del Tema Oscuro Gaming

⚠ Implementar diseño gaming visualmente consistente sin que múltiples colores de acento resultaran caóticos.

✓ ThemeData centralizado en main.dart con colores constantes (cyan, rojo, verde, naranja) declarados en app_constants.dart y aplicados sistemáticamente.

CRITERIOS DE EVALUACIÓN — CUMPLIMIENTO

✓	**Navegación**	Named Routes + onGenerateRoute + 5+ pantallas	10
✓	**Widgets y Formularios**	ListView, GridView, TextField, RadioListTile, PopupMenuButton	15
✓	**Persistencia (Hive + SharedPrefs)**	3 boxes Hive + 5 claves SharedPreferences	20
✓	**Consumo de API**	RAWG API con paginación, caché y manejo de errores	10
✓	**Animaciones**	AnimatedContainer, ScaleTransition, AnimatedOpacity stagger	10
✓	**Publicación Google Play**	APK firmado con ícono personalizado y versionado	15
✓	**Documentación**	22 páginas con arquitectura, código fuente y retos	10
✓	**Presentación**	Demo en vivo + fl	[illegible]
		TOTAL	**100 pts**

CONCLUSIONES Y PRÓXIMOS PASOS

Aprendizajes del Equipo

- Navegación Named Routes y paso de argumentos
- Manejo de estado con StatefulWidget y setState
- Consumo de API REST paginada con http
- Persistencia con Hive (null safety, TypeAdapters)
- SharedPreferences para preferencias del usuario
- Animaciones ScaleTransition y AnimatedContainer
- Diseño gaming coherente con ThemeData centralizado

Mejoras Futuras

Firebase Auth	Autenticación de usuarios
Reseñas	Sistema de calificaciones por usuario
Stripe / MercadoPago	Pasarela de pago real
Admin Panel	Gestión del catálogo y estadísticas
Push Notifications	Firebase Cloud Messaging para ofertas
iOS Support	Soporte multiplataforma Apple
Light Theme	Modo claro como alternativa

🎮 GAME VAULT

¡Gracias!

Videojuegos Disada

D

Daniel Martínez Sánchez

Desarrollo & Arquitectura

J

José Salvador Montes Luna

UI/UX & Tienda

D

Diego Eduardo Hurtado Hernández

API & Persistencia